

Homework: MySQL on VM vs Managed Service (SQLAlchemy + pandas)

1) Overview

You will provision two MySQL databases on the same cloud: (A) a VM you harden and configure yourself, and (B) the cloud's managed MySQL offering. You will then connect to both using SQLAlchemy in Python, create a new database and table with pandas, insert data, and read it back. You will document steps, timing, and the differences in setup effort and operational friction.

Managed service names by cloud

- **Azure:** Azure Database for MySQL – Flexible Server
- **GCP:** Cloud SQL for MySQL
- **OCI:** MySQL Database Service (MDS)

VM services by cloud

- **Azure:** Azure Virtual Machines
 - **GCP:** Compute Engine
 - **OCI:** Compute
-

2) Learning Objectives

- Contrast operational tasks between self-managed and managed MySQL.
 - Configure network access, and users.
 - Use SQLAlchemy to connect, create a database/table, and round-trip data with pandas.
 - Capture evidence (screens/notes) and compare time-to-first-query and setup difficulty.
-

3) Packages that you will use:

- Inside of your requirements will need: `sqlalchemy`, `pymysql` (or `mysqlclient`), `pandas`, `python-dotenv`
-

4) Repository & Deliverables

Create a repo named `HHA504_mysql_vm_vs_managed` with this structure:

```
HHA504_mysql_vm_vs_managed/  
├── README.md  
├── .gitignore  
├── .env.example  
├── scripts/  
│   ├── vm_demo.py  
│   └── managed_demo.py  
└── sql/
```

Make sure to ignore your .env
Do NOT commit real secrets

SQLAlchemy+pandas against VM MySQL
SQLAlchemy+pandas against managed MySQL

```

|   └─ init.sql                # Optional: user/db bootstrap you ran on
VM
|   └─ screenshots/
|       └─ vm/                # VM portal, firewall, daemon status,
CLI, etc.
|       └─ managed/          # Managed service creation, connection
details
└─ docs/
    └─ setup_notes_vm.md
    └─ setup_notes_managed.md
    └─ comparison.md          # timing & difficulty comparison

```

Deliverables (must be in **main**):

- **README.md** with:
 - Cloud chosen and region
 - Public high-level steps to reproduce (bullets)
 - Connection string patterns (no secrets), and how you stored secrets locally
 - Screenshots summary and links
- **scripts/vm_demo.py** and **scripts/managed_demo.py** that (you can just copy and paste the provided files, and or modify them as you like):
 1. Read credentials from environment (**.env**)
 2. Connect to MySQL via SQLAlchemy
 3. Create a new database (e.g., **dummydb**) if not exists
 4. Create a table (e.g., **visits**) via pandas **to_sql**
 5. Insert 5–10 rows from a pandas DataFrame
 6. Read back with **pd.read_sql** and print row count
- **docs/setup_notes_vm.md** and **docs/setup_notes_managed.md**:
 - Ordered steps you executed, with command snippets
 - Any config files you edited (e.g., **mysqld.cnf**), with the exact lines
 - Troubles you hit and how you solved them
 - **Start-to-finish elapsed time** (minutes) measured by you
- **docs/comparison.md**:
 - A short paragraph on which you would choose in production and why
- **screenshots/** evidence:
 - VM: portal/console creation page, package install, service status, firewall rules/security group, MySQL prompt proving DB/table exist, and a local or bastion test
 - Managed: creation wizard summary, connection info, authorized networks/VPC, connectivity test, metrics/overview page
- 2–4 minute **recording** (Zoom/Loom link in README) showing:

- Your repo
 - Running each script end-to-end (VM then Managed) and printed results
-

5) Task A — MySQL on a VM (Self-managed)

1. Provision VM

- OS: Ubuntu LTS or Oracle Linux
- Instance type: small (e.g., 1–2 vCPU, 2–4 GB RAM)
- Open **TCP ports 22 and 3306**

2. Install & Configure MySQL

- Install server packages; enable and start service.
- Set strong root password (or auth plugin).
- Edit `mysqld.cnf` (bind-address), restart service.
- Configure firewall/security group rules minimally; note your choices in `setup_notes_vm.md`.

3. **Test Locally or Inside of Cloud Console (VS Code in GCP) environment **

- `mysql -u <user> -p -h 127.0.0.1 -P 3306` from VM
 - Optional: set up an SSH tunnel from your dev machine instead of opening 3306 publicly.
-

6) Task B — Managed MySQL

Create the provider's managed MySQL with a small tier. Capture:

- Engine version, vCPU/RAM tier
 - Network model (public IP allowlist)
 - Initial admin user, DB name
 - Any automatic backups/HA configuration chosen
-

7) Python: SQLAlchemy + pandas Template

Create a `.env` (do **not** commit) from `.env.example`:

```
# VM
VM_DB_HOST=10.0.1.10      # or 127.0.0.1 via SSH tunnel
VM_DB_PORT=3306
VM_DB_USER=class_user
VM_DB_PASS=change_me
VM_DB_NAME=class_db_netid

# Managed
MAN_DB_HOST=...
MAN_DB_PORT=3306
```

```
MAN_DB_USER=class_user
MAN_DB_PASS=change_me
MAN_DB_NAME=class_db_netid
```

Connection URL patterns (choose *one* driver and be consistent):

- PyMySQL: `mysql+pymysql://USER:PASS@HOST:PORT/DBNAME`
- mysqlclient: `mysql+mysql://USER:PASS@HOST:PORT/DBNAME`

Snippet (adapt for vm vs managed):

```
# scripts/vm_demo.py (similar for managed_demo.py)
```

Notes

- You must use a public IP (for demo): remember to delete resources after submission. They are expensive.

8) What to Capture (Screenshots)

- **VM path:** VM creation summary, security group/firewall, MySQL install + `systemctl status mysql`, `mysql --version`, `mysql` prompt showing `SHOW DATABASES;` and your DB/table present.
- **Managed path:** service creation summary, connection endpoints, network/authorized list, and a simple query window or metrics page.
- **Python runs:** terminal output of each script printing row counts, plus your environment layout (no secrets).

9) Comparison ([docs/comparison.md](#))

Write approximately 200-400 words concluding which you'd choose in production for: (a) a small student app; (b) a departmental analytics DB; (c) a HIPAA-aligned workload (assume a BAA is available in your cloud).

10) Safety & Clean-up

- Delete public ingress rules you created.
 - Stop or delete the VM and managed instance after you finish to avoid costs.
-